

# C++ Network Code Bug Review

<https://gemini.google.com/app/6c4e7d98a5c7a1fb>

```
User prompt: #include "tlsfp/capture.hpp" #include "tlsfp/parser.hpp" #include <iostream>
#include <netinet/ip.h> #include <netinet/ip6.h> #include <netinet/tcp.h> #include
<netinet/in.h> #include <arpa/inet.h> namespace tlsfp { static pcap_t *g_pcap_handle =
nullptr; void signal_handler(int signum) { if (g_pcap_handle != nullptr) { std::cout << "\n[*]
Caught signal " << signum << ", stopping capture gracefully...\n";
pcap_breakloop(g_pcap_handle); } } void packet_callback(u_char *user_data, const struct
pcap_pkthdr *pkthdr, const u_char *packet) { CaptureContext *ctx =
reinterpret_cast<CaptureContext*>(user_data); if (!ctx) return; // 1. Resolve Link-Layer Offset
dynamically size_t link_header_len = 0; switch (ctx->link_type) { case DLT_EN10MB: //
Standard Ethernet / Wi-Fi (14 bytes) link_header_len = 14; break; case DLT_LINUX_SLL2: //
Linux Cooked v2 (WSL / tcpdump -i any) (20 bytes) link_header_len = 20; break; case
DLT_LINUX_SLL: // Linux Cooked v1 (16 bytes) link_header_len = 16; break; case
DLT_NULL: // BSD / macOS Loopback (4 bytes) link_header_len = 4; break; case DLT_RAW:
// Raw IP (VPN / tun interfaces) (0 bytes) link_header_len = 0; break; default: return; //
Unsupported link-layer } // Ensure packet has at least 1 byte after link layer to check the IP
version nibble if (pkthdr->caplen < link_header_len + 1) { return; } // 2. Determine IP Version
(first 4 bits of the IP header) uint8_t ip_version = (packet[link_header_len] >> 4); size_t
ip_header_len = 0; char src_ip[INET6_ADDRSTRLEN]; char dst_ip[INET6_ADDRSTRLEN];
FlowKey key{}; if (ip_version == 4) { // --- IPv4 Path --- if (pkthdr->caplen < link_header_len +
sizeof(struct ip)) return; const struct ip *ip_hdr = reinterpret_cast<const struct ip *>(packet +
link_header_len); if (ip_hdr->ip_p != IPPROTO_TCP) return; // Verify TCP protocol
ip_header_len = ip_hdr->ip_hl * 4; if (ip_header_len < 20) return; inet_ntop(AF_INET, &
(ip_hdr->ip_src), src_ip, INET_ADDRSTRLEN); inet_ntop(AF_INET, &(ip_hdr->ip_dst),
dst_ip, INET_ADDRSTRLEN); // Store binary IP addresses directly (Zero Heap Allocation)
key.ip_version = 4; key.src_ip.v4 = ip_hdr->ip_src; key.dst_ip.v4 = ip_hdr->ip_dst; } else if
(ip_version == 6) { // --- IPv6 Path --- if (pkthdr->caplen < link_header_len + sizeof(struct
ip6_hdr)) return; const struct ip6_hdr *ip6_hdr = reinterpret_cast<const struct ip6_hdr *>
(packet + link_header_len); if (ip6_hdr->ip6_nxt != IPPROTO_TCP) return; // Verify TCP
protocol ip_header_len = sizeof(struct ip6_hdr); // Fixed 40 bytes for standard IPv6
inet_ntop(AF_INET6, &(ip6_hdr->ip6_src), src_ip, INET6_ADDRSTRLEN);
inet_ntop(AF_INET6, &(ip6_hdr->ip6_dst), dst_ip, INET6_ADDRSTRLEN); // Store binary IP
addresses directly (Zero Heap Allocation) key.ip_version = 6; key.src_ip.v6 = ip6_hdr-
>ip6_src; key.dst_ip.v6 = ip6_hdr->ip6_dst; } else { return; // Non-IP packet } // 3. Inspect TCP
Header if (pkthdr->caplen < link_header_len + ip_header_len + sizeof(struct tcphdr)) { return;
} const struct tcphdr *tcp_hdr = reinterpret_cast<const struct tcphdr *>(packet +
link_header_len + ip_header_len); size_t tcp_header_len = tcp_hdr->th_off * 4; if
(tcp_header_len < 20) { return; } // Set 5-tuple ports key.src_port = tcp_hdr->th_sport;
key.dst_port = tcp_hdr->th_dport; // 4. Calculate Payload Offset & Length size_t
header_total_len = link_header_len + ip_header_len + tcp_header_len; if (pkthdr->caplen <=
header_total_len) { return; // Pure TCP ACK/SYN/FIN or header-only frame } size_t
payload_len = pkthdr->caplen - header_total_len; const uint8_t *payload = packet +
header_total_len; // 5. TCP Sequence Alignment and Deduplication StreamBuffer &buf = ctx-
>active_flows[key]; buf.last_seen = pkthdr->ts.tv_sec; uint32_t seq = ntohl(tcp_hdr->th_seq);
if (!buf.seq_initialized) { buf.next_seq = seq; buf.seq_initialized = true; } if (seq <
buf.next_seq) { // Handle duplicate bytes / retransmissions uint32_t diff = buf.next_seq - seq;
if (diff >= payload_len) return; // Entirely duplicate, discard payload += diff; payload_len -= diff;
} else if (seq > buf.next_seq) { // Gap detected due to out-of-order packet: clear flow state ctx-
>active_flows.erase(key); return; } if (buf.len + payload_len > StreamBuffer::MAX_BUF_SIZE)
{ ctx->active_flows.erase(key); return; } std::memcpy(buf.bytes + buf.len, payload,
payload_len); buf.len += static_cast<uint16_t>(payload_len); buf.next_seq += payload_len; //
```

```

Advance expected sequence number // 6. Inspect TLS Record Header if (buf.len < 5) { return;
// Wait for full 5-byte TLS record header } uint8_t record_type = buf.bytes[0]; uint8_t
major_ver = buf.bytes[1]; uint8_t minor_ver = buf.bytes[2]; // Declared here so it remains in
scope for validation below uint16_t tls_rec_len = (static_cast<uint16_t>(buf.bytes[3]) << 8) |
buf.bytes[4]; bool is_valid_tls = (record_type == 0x16) && (major_ver == 0x03) && (minor_ver
<= 0x04) && (tls_rec_len <= 16384); if (!is_valid_tls) { ctx->active_flows.erase(key); return; }
size_t total_expected_len = 5 + tls_rec_len; if (buf.len < total_expected_len) { return; // Wait
for full TLS record payload } if (tls_rec_len < 4 || buf.len < 9) { ctx->active_flows.erase(key);
return; } // Process Complete Reassembled ClientHello (0x01) or ServerHello (0x02) uint8_t
handshake_type = buf.bytes[5]; if (handshake_type == 0x01 || handshake_type == 0x02) {
uint32_t hs_len = (static_cast<uint32_t>(buf.bytes[6]) << 16) | (static_cast<uint32_t>
(buf.bytes[7]) << 8) | static_cast<uint32_t>(buf.bytes[8]); // Validate handshake length against
tls_rec_len if (4 + hs_len > tls_rec_len) { ctx->active_flows.erase(key); return; } if (ctx-
>dumper != nullptr) { pcap_dump(reinterpret_cast<u_char*>(ctx->dumper), pkthdr, packet);
pcap_dump_flush(ctx->dumper); } uint16_t src_port = ntohs(tcp_hdr->th_sport); uint16_t
dst_port = ntohs(tcp_hdr->th_dport); std::cout << "[+] Captured " << (handshake_type ==
0x01 ? "ClientHello" : "ServerHello") << " | Flow: [" << src_ip << ":" << src_port << " -> [" <<
dst_ip << ":" << dst_port << " | Reassembled Size: " << buf.len << " bytes" << " | Handshake
Payload: " << hs_len << " bytes\n"; ctx->active_flows.erase(key); } else { ctx-
>active_flows.erase(key); } } bool start_capture(const CaptureOptions &opts) { char
errbuf[PCAP_ERRBUF_SIZE]; pcap_t *handle = nullptr; if (!opts.read_filename.empty()) {
handle = pcap_open_offline(opts.read_filename.c_str(), errbuf); } else { handle =
pcap_open_live(opts.interface_name.c_str(), BUFSIZ, 1, 1000, errbuf); } if (!handle) { std::cerr
<< "[-] Error opening pcap target: " << errbuf << "\n"; return false; } g_pcap_handle = handle;
struct bpf_program fp; if (pcap_compile(handle, &fp, "tcp port 443", 0,
PCAP_NETMASK_UNKNOWN) == 0) { pcap_setfilter(handle, &fp); pcap_freecode(&fp); }
else { std::cerr << "[-] Warning: Failed to set BPF filter 'tcp port 443'\n"; } CaptureContext ctx;
ctx.link_type = pcap_datalink(handle); std::cout << "[*] Capture initialized. Datalink type: " <<
ctx.link_type << "\n"; if (!opts.write_filename.empty()) { ctx.dumper =
pcap_dump_open(handle, opts.write_filename.c_str()); if (!ctx.dumper) { std::cerr << "[-]
Warning: Could not open output PCAP for writing: " << opts.write_filename << "\n"; } }
pcap_loop(handle, 0, packet_callback, reinterpret_cast<u_char*>(&ctx)); if (ctx.dumper) {
pcap_dump_flush(ctx.dumper); pcap_dump_close(ctx.dumper); pcap_close(handle);
g_pcap_handle = nullptr; std::cout << "[*] Capture completed.\n"; return true; } } // namespace
tlsfp #ifndef TLSFP_CAPTURE_HPP #define TLSFP_CAPTURE_HPP #include <string>
#include <pcap.h> #include <cstring> #include <unordered_map> #include <cstdint>
namespace tlsfp { struct CaptureOptions { std::string interface_name; std::string
read_filename; std::string write_filename; }; struct FlowKey { uint8_t ip_version; union { struct
in_addr v4; struct in6_addr v6; } src_ip, dst_ip; uint16_t src_port; uint16_t dst_port; bool
operator==(const FlowKey &o) const { if (ip_version != o.ip_version || src_port != o.src_port ||
dst_port != o.dst_port) return false; if (ip_version == 4) { return src_ip.v4.s_addr ==
o.src_ip.v4.s_addr && dst_ip.v4.s_addr == o.dst_ip.v4.s_addr; } return
std::memcmp(&src_ip.v6, &o.src_ip.v6, sizeof(struct in6_addr)) == 0 &&
std::memcmp(&dst_ip.v6, &o.dst_ip.v6, sizeof(struct in6_addr)) == 0; } }; struct FlowHash {
std::size_t operator()(const FlowKey &k) const { std::size_t h = k.ip_version ^
(static_cast<std::size_t>(k.src_port) << 16) ^ k.dst_port; if (k.ip_version == 4) { h ^=
k.src_ip.v4.s_addr ^ (k.dst_ip.v4.s_addr * 0x9e3779b9); } else { const uint64_t *p1 =
reinterpret_cast<const uint64_t*>(&k.src_ip.v6); const uint64_t *p2 = reinterpret_cast<const
uint64_t*>(&k.dst_ip.v6); h ^= p1[0] ^ p1[1] ^ p2[0] ^ p2[1]; } return h; } }; struct StreamBuffer {
static constexpr size_t MAX_BUF_SIZE = 16389; // 5-byte TLS Header + 16384 Max Payload
uint8_t bytes[MAX_BUF_SIZE]; uint16_t len = 0; // TCP State Tracking uint32_t next_seq = 0;
bool seq_initialized = false; time_t last_seen = 0; }; struct CaptureContext { pcap_dumper_t
*dumper{nullptr}; int link_type{0}; std::unordered_map<FlowKey, StreamBuffer, FlowHash>
active_flows; }; bool start_capture(const CaptureOptions &opts); void packet_callback(u_char

```

*\*user\_data, const struct pcap\_pkthdr \*pkthdr, const u\_char \*packet); void signal\_handler(int signum); } #endif tell me if this code is correct and will work*

Response: The provided C++ code will compile with minor header organization, but **it is not fully correct and will fail or crash in production network environments**. It contains critical logic flaws in TCP stream reassembly, TLS parsing, memory management, and signal handling.

| Severity | Component       | Issue Description                                                                                       |
|----------|-----------------|---------------------------------------------------------------------------------------------------------|
| Critical | TCP Reassembly  | Erases flows on valid traffic if the TCP <code>SYN</code> frame is captured                             |
| Critical | Packet Offsets  | Includes Ethernet padding bytes into the TCP payload                                                    |
| High     | TLS Parsing     | Erases flows when TLS Handshake messages span multiple TLS records                                      |
| High     | Memory Leak     | No garbage collection for abandoned or incomplete TCP connections                                       |
| Medium   | IPv6 Handling   | Fails to parse TCP packets containing IPv6 Extension Headers                                            |
| Medium   | Signal Handling | <code>signal_handler</code> is defined but never registered with OS signals                             |
| Low      | Strict Aliasing | <code>reinterpret_cast&lt;const uint64_t*&gt;</code> in <code>FlowHash</code> causes Undefined Behavior |

## 1. TCP SYN Sequence Number Bug

When capturing a flow from the TCP `SYN` packet, TCP sequence logic dictates that the `SYN` flag consumes 1 sequence number despite having a payload length of 0.

In your current logic:

- Packet 1 (`SYN`): `next_seq` is initialized to `seq`. Payload is 0, so `next_seq` remains `seq`.
- Packet 2 (`DATA`): `seq` is `SYN_seq + 1`.
- Because `seq > buf.next_seq`, the condition `else if (seq > buf.next_seq)` triggers and **erases the flow immediately**.

## 2. Ethernet Padding Contamination

Calculating payload length via `pkthdr->caplen - header_total_len` is unsafe. Ethernet frames require a minimum frame length of 64 bytes (60 bytes without FCS). Small TCP packets (such as early handshake frames) are padded with trailing zeros by network adapters.

Using `caplen` causes these padding bytes to be copied into `buf.bytes` as corrupt TLS payload data. The actual payload length must be derived from the IP layer total length header (`ntohs(ip_hdr->ip_len)` for IPv4).

### 3. Multi-Record TLS Handshake Failures

Your code assumes a complete TLS Handshake message always fits within a single TLS record:

```
if (4 + hs_len > tls_rec_len) {
    ctx->active_flows.erase(key);
    return;
}
```

If a `ClientHello` or `ServerHello` spans multiple TLS records (common with large Post-Quantum key exchanges, long extension lists, or large TLS certificates), `hs_len` will legitimately exceed `tls_rec_len`. Your code explicitly drops these valid connections.

### 4. Unbounded Memory Leak in `active_flows`

The `std::unordered_map<FlowKey, StreamBuffer>` grows indefinitely. If a TCP flow drops, timed out, finishes without matching TLS criteria, or port 443 traffic is non-TLS (e.g., SSH or VPN over 443), its `StreamBuffer` (~16 KB) remains in memory permanently. You need an active sweep or timeout mechanism based on `last_seen`.

### 5. Unregistered Signal Handler

`signal_handler()` is defined in `capture.cpp`, but it is never registered with the system signal table. Pressing `Ctrl+C` will terminate the program instantly without calling `pcap_breakloop()` or flushing open PCAP dump files.

You must register the handler inside `start_capture()`:

```
#include <csignal>

// Inside start_capture():
std::signal(SIGINT, signal_handler);
std::signal(SIGTERM, signal_handler);
```

### 6. IPv6 Extension Headers Dropped

In IPv6, `ip6_hdr->ip6_nxt` does not always equal `IPPROTO_TCP`. IPv6 packets often pass through Extension Headers (such as Hop-by-Hop, Routing, or Fragment headers). Checking `ip6_nxt != IPPROTO_TCP` causes the parser to discard valid TCP packets wrapped in IPv6 extensions.

### 7. Strict Aliasing Violation in `FlowHash`

```
const uint64_t *p1 = reinterpret_cast<const uint64_t*>(&k.src_ip.v6);
```

Reinterpreting `struct in6_addr` (an array of bytes/shorts) as `uint64_t*` violates C++ strict aliasing rules and can cause unaligned memory access crashes on non-x86 architectures (e.g., ARM). Use `std::memcpy` or process the bytes via standard standard

integer types instead.

---